Backport of the F2FS *max_write_bio_folio_cnt* series to Linux 6.18.2

Point-by-point description of every change made

Series by Chao Yu (posted 17 Oct 2026) · backport prepared 4 Nov 2026

Source series	[PATCH 1/2] f2fs: introduce max_write_bio_{folio,entry}_cnt module parameters [PATCH 2/2] f2fs: remove max_atc_write_bio_size
Target	vanilla linux-6.18.2 (tarball supplied), fs/f2fs only
Delivered	4 patches: 2 reconstructed groundwork + 2 adapted from the upstream series, plus a combined diff and README
Net change	4 files: compress.c +1/−1 · data.c +57/−20 · f2fs.h +20/−3 · super.c +4/−4 (total +82/−28)
Status	applies cleanly · compiles (gcc 13.3, W=1) with no warnings · **not booted / not run-tested**

Summary

1. The upstream series cannot be applied to 6.18.2: 6 of 9 data.c hunks and the f2fs.h hunk of patch 1/2 fail, and every hunk of 2/2 fails or is ignored. The code it builds on (async write completion, `struct f2fs_bio`, metadata cache) does not exist in 6.18.2.
2. The backport is **reduced**: only the folio limit is kept, the metadata-cache parts are dropped.
3. Because the async-completion groundwork was not available, patches 1 and 2 **re-create the minimum** of it; they are not upstream commits.
4. Patches 3 and 4 are the upstream 1/2 and 2/2, adapted (same diffstat for 2/2).
5. Default behaviour is unchanged (limit = UINT_MAX = never offload); new fixed costs are one workqueue per mount and a 40-byte per-bio front pad.

1. Starting point — why the series does not apply to 6.18.2

Both upstream patches were test-applied (`patch -p1 --dry-run`) to the pristine 6.18.2 tree, each on its own. Results:

Upstream patch / file	Hunks	Result on 6.18.2
1/2 fs/f2fs/data.c	9	**6 FAILED** (#1 #3 #4 #6 #7 #9). #2, #5, #8 apply only with fuzz/offset (−76, −161, −224 lines) and would not compile without the missing symbols
1/2 fs/f2fs/f2fs.h	1	**FAILED** — the `struct f2fs_bio` context it patches does not exist
2/2 Documentation/ABI/…/sysfs-fs-f2fs	1	**FAILED** — the ABI entry it removes is not in 6.18.2
2/2 fs/f2fs/data.c	1	**FAILED**
2/2 f2fs.h, super.c, sysfs.c	1 + 1 + 2	all **ignored** (“reversed or already applied”): they delete lines that were never added

Cause: symbols the series relies on do not exist in the stock 6.18.2 f2fs sources (occurrence count in fs/f2fs/*.[ch]):

Symbol	6.18.2	Needed by
`max_atc_write_bio_size`	0	1/2 (condition) and 2/2 (removal)
`f2fs_write_end_io_work`	0	1/2 hunks #2, #3
`struct f2fs_bio`	0	1/2 f2fs.h hunk, every `F2FS_BIO()` use
`f2fs_is_cache_bio, f2fs_cached_block, f2fs_submit_cache_write`	0	metadata-cache series
`f2fs_io_type_to_write_stream, bi_write_stream`	0	1/2 hunk #4 context
`ffs_entry_slab`	0	1/2 hunk #1 context
`sbi->wq`	0	workqueue used for the offload
`post_read_wq`	14	exists, but see below

What stock 6.18.2 actually does (read from the source)

- **Write completion is fully synchronous.** `f2fs_write_end_io()` is one function: unbind iostat ctx, fault injection, a loop that ends writeback for every folio, wake `cp_wait`, `bio_put()`.
- **No per-bio state.** `f2fs_bioset` is created with `front_pad = 0`; there is no place to put a `work_struct`.
- **The workqueue is conditional.** `sbi->post_read_wq` is allocated only when the filesystem has encryption, verity or compression; on a plain mount it is NULL, so queueing write completion to it would crash.
- **Write bios are built in four places** (all of which must count entries): `f2fs_submit_page_bio()`, `add_bio_entry()`, `add_ipu_page()`, `f2fs_submit_page_write()`; allocation is in `__bio_alloc()`. In 6.18.2 the first three of these partly use page-based `bio_add_page()`, not the folio calls the upstream context expects.

2. Backport strategy — decisions taken

#	Decision	Reason
D1	Mirror the upstream layering as separate patches: workqueue → async completion → this series (1/2) → removal of the old knob (2/2).	Each step is reviewable and bisectable; patches 3 and 4 stay close to the real upstream diffs, so a later move to the real series is easier.
D2	**Reduce the scope:** keep only `max_write_bio_folio_cnt`; drop `max_write_bio_entry_cnt`, `f2fs_is_cache_bio()`, the `f2fs_submit_cache_write()` hunk and the `entry` member.	They belong to the metadata-cache series, which 6.18.2 does not have and which is not needed to bound folio-bio completion.
D3	**Re-create the async-completion groundwork** (patches 1–2) instead of waiting for the upstream commits.	The original commits were not available. Only what the series needs was rebuilt; the missing pieces are listed in §5.
D4	Keep the `max_atc_write_bio_size` sysfs knob in patch 2 and remove it again in patch 4.	Patch 4 then matches upstream 2/2 line for line (5 files, +2/−17), and the tree passes through the same intermediate state as upstream.
D5	Names: `sbi->wq`, workqueue “f2fs_wq”, `f2fs_{init,destroy}_wq()`.	Inferred from the upstream commit message (“sbi->wq”). May differ from the real series — expect a trivial conflict when switching.
D6	Authorship: patches 3–4 carry Chao Yu as author and his Signed-off-by plus a “[Backport …]” note; patches 1–2 are marked “reconstructed, not an upstream commit”.	Honest provenance; add your own Signed-off-by when you submit or carry them.

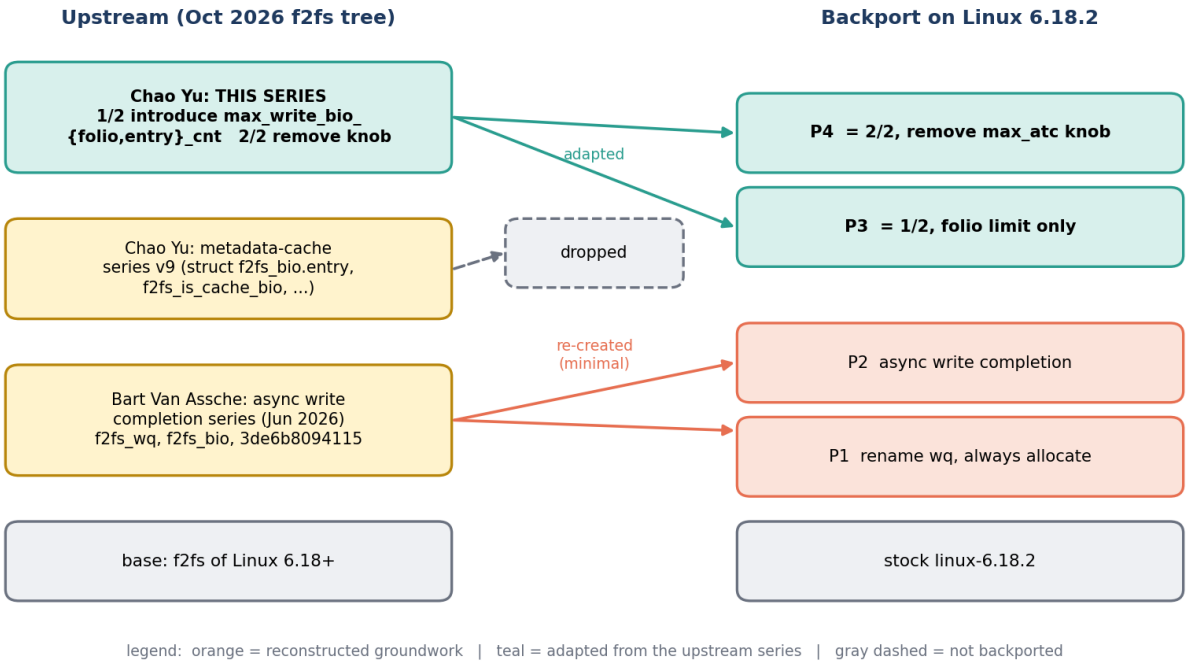


Figure 1 — Mapping of the upstream series to the four backport patches.

3. Patch-by-patch changes

Patch 1 — f2fs: rename f2fs_post_read_wq to f2fs_wq and always allocate it

Origin	reconstructed groundwork (stands in for the workqueue part of the upstream async-completion series)
Files	compress.c +1/−1 · data.c +8/−14 · f2fs.h +3/−3 · super.c +4/−4
Purpose	Give every mounted f2fs a workqueue that write completion can use. Pure rename for the read path.

1.1 **f2fs.h — struct f2fs_sb_info:** field `post_read_wq` renamed to `wq`; comment now "f2fs workqueue (post read, write end io)".

1.2 **f2fs.h — prototypes:** `f2fs_init_post_read_wq()` / `f2fs_destroy_post_read_wq()` renamed to `f2fs_init_wq()` / `f2fs_destroy_wq()`.

1.3 **data.c — f2fs_init_wq():** removed the early return for filesystems without encrypt/verity/compression, so the queue is **always** created: `alloc_workqueue("f2fs_wq", WQ_UNBOUND | WQ_HIGHPRI, num_online_cpus())`; still returns −ENOMEM on failure.

1.4 **data.c — f2fs_destroy_wq():** renamed; same `destroy_workqueue()` guarded by a NULL check.

1.5 **Read-path users updated:** `f2fs_read_end_io()` (data.c) and the decompress free path (compress.c) now queue to `sbi->wq`.

1.6 **super.c — f2fs_fill_super():** init call renamed, error message now "Failed to initialize workqueue"; the two destroy call sites (mount error path and `f2fs_put_super()`) renamed.

Effect: behaviour of reads is unchanged; every mount now owns one unbound, high-priority workqueue (small memory cost).

Patch 2 — f2fs: Run f2fs_write_end_io() asynchronously

Origin	reconstructed groundwork; **not** upstream commit 3de6b8094115 (same idea, minimal re-implementation)
Files	ABI doc +10 · data.c +30/−6 · f2fs.h +17 · super.c +1 · sysfs.c +2
Purpose	Allow a write bio to be completed from a workqueue when `f2fs_write_end_io()` runs in atomic context.

2.1 **f2fs.h — new struct f2fs_bio** (placed before struct f2fs_sb_info): `{ struct work_struct work; struct bio bio; }` plus accessor `F2FS_BIO(bio) = container_of()`. It lives in the bioset front pad, i.e. in front of every bio allocated from `f2fs_bioset`.

2.2 **f2fs.h — struct f2fs_sb_info:** new `u32 max_atc_write_bio_size` (largest write bio completed in atomic context).

2.3 **data.c — f2fs_init_bioset():** `front_pad` changed from 0 to `offsetof(struct f2fs_bio, bio)` so the work item is allocated with each bio. Read bios allocated from the same bioset get the pad too.

2.4 **data.c — split of f2fs_write_end_io():** the completion loop becomes `f2fs_write_end_bio()` (fault injection, per-folio end-writeback, wake `cp_wait`, `bio_put()`). It now takes `sbi` from `bio->bi_private`.

2.5 **data.c — f2fs_write_end_io_work():** new worker; recovers the bio with `container_of(work, struct f2fs_bio, work)` and calls `f2fs_write_end_bio()`.

2.6 **data.c — new f2fs_write_end_io() (the bi_end_io):** first `iostat_update_and_unbind_ctx(bio)` (moved here so `bi_private` is the sbi again before any deferral), then: if `in_atomic()` and `bi_size > sbi->max_atc_write_bio_size`, `INIT_WORK() + queue_work(sbi->wq, …)` and return; otherwise call `f2fs_write_end_bio()` inline. The zoned wrapper `f2fs_zone_write_end_io()` is untouched: it restores `bi_private` and calls this function.

2.7 **super.c — f2fs_fill_super():** `sbi->max_atc_write_bio_size = UINT_MAX` (feature off by default).

2.8 **sysfs.c:** `F2FS_SBI_RW_ATTR(max_atc_write_bio_size, …)` and its `ATTR_LIST` entry.

2.9 **ABI doc:** entry for `/sys/fs/f2fs/<disk>/max_atc_write_bio_size` appended at the end of sysfs-fs-f2fs (text taken from the removal hunk of upstream 2/2).

```
static void f2fs_write_end_io(struct bio *bio)
{
    struct f2fs_sb_info *sbi;

    iostat_update_and_unbind_ctx(bio);
    sbi = bio->bi_private;

    if (in_atomic() && bio->bi_iter.bi_size > sbi->max_atc_write_bio_size) {
        struct work_struct *w = &F2FS_BIO(bio)->work;

        INIT_WORK(w, f2fs_write_end_io_work);
        queue_work(sbi->wq, w);
        return;
    }
    f2fs_write_end_bio(bio);
}
```

Patch 3 — f2fs: introduce max_write_bio_folio_cnt module parameter

Origin	upstream 1/2 by Chao Yu, **reduced** (folio limit only) and re-based onto 6.18.2 code
Files	data.c +20/−1 · f2fs.h +3/−1 (upstream: 2 files, +28/−1)
Purpose	Replace the unreliable byte test with an exact per-bio folio count compared with a global read-only module parameter.

3.1 **f2fs.h — struct f2fs_bio:** new member `unsigned int entry_cnt` (between `work` and `bio`); comment explains it.

3.2 **data.c — module parameter:** `static unsigned int max_write_bio_folio_cnt = UINT_MAX;` `module_param(…, uint, 0444)` with `MODULE_PARM_DESC`. Read-only at runtime; set on the kernel command line (`f2fs.max_write_bio_folio_cnt=N`) or as a modprobe option.

3.3 **data.c — f2fs_max_write_bio_cnt(bio):** helper kept in the upstream shape but returns only the folio limit (upstream chooses between the folio and the entry limit using `f2fs_is_cache_bio()`).

3.4 **data.c — f2fs_write_end_io():** offload condition extended to `in_atomic() && (bi_size > max_atc_write_bio_size || entry_cnt > f2fs_max_write_bio_cnt(bio))` (the old test is removed in patch 4).

3.5 **data.c — __bio_alloc():** `F2FS_BIO(bio)->entry_cnt = 0` in the write branch.

3.6 **data.c — counting, four sites** (this is where the 6.18.2 code differs from the upstream context): `f2fs_submit_page_bio()` after `bio_add_folio_nofail()`, only if not a read; `add_bio_entry()` after `bio_add_page()`; `add_ipu_page()` after a successful `bio_add_page()` (inside `bio_list_lock`); `f2fs_submit_page_write()` after the folio fits in the bio (inside `io_rwsem`). Plain `entry_cnt++` is enough because the bio is only modified under those locks before submission.

3.7 **Dropped from upstream 1/2:** the second parameter, `f2fs_is_cache_bio()`, the `f2fs_submit_cache_write()` increment, and the `entry` member/context in f2fs.h.

Patch 4 — f2fs: remove max_atc_write_bio_size

Origin	upstream 2/2 by Chao Yu, adapted (ABI entry is removed from the end of the file)
Files	ABI doc −10 · data.c +2/−2 · f2fs.h −2 · super.c −1 · sysfs.c −2 (identical diffstat to upstream: 5 files, +2/−17)
Purpose	Drop the superseded per-filesystem byte knob and the dead `bi_size` test.

4.1 **data.c:** condition simplified to `in_atomic() && F2FS_BIO(bio)->entry_cnt > f2fs_max_write_bio_cnt(bio)`.

4.2 **f2fs.h:** removed the `max_atc_write_bio_size` field and its comment.

4.3 **super.c:** removed its initialisation in `f2fs_fill_super()`.

4.4 **sysfs.c:** removed the `F2FS_SBI_RW_ATTR` and `ATTR_LIST` lines.

4.5 **ABI doc:** removed the whole `max_atc_write_bio_size` entry. After this patch /sys/fs/f2fs/<disk>/max_atc_write_bio_size no longer exists (ENOENT).

4. Resulting code structure

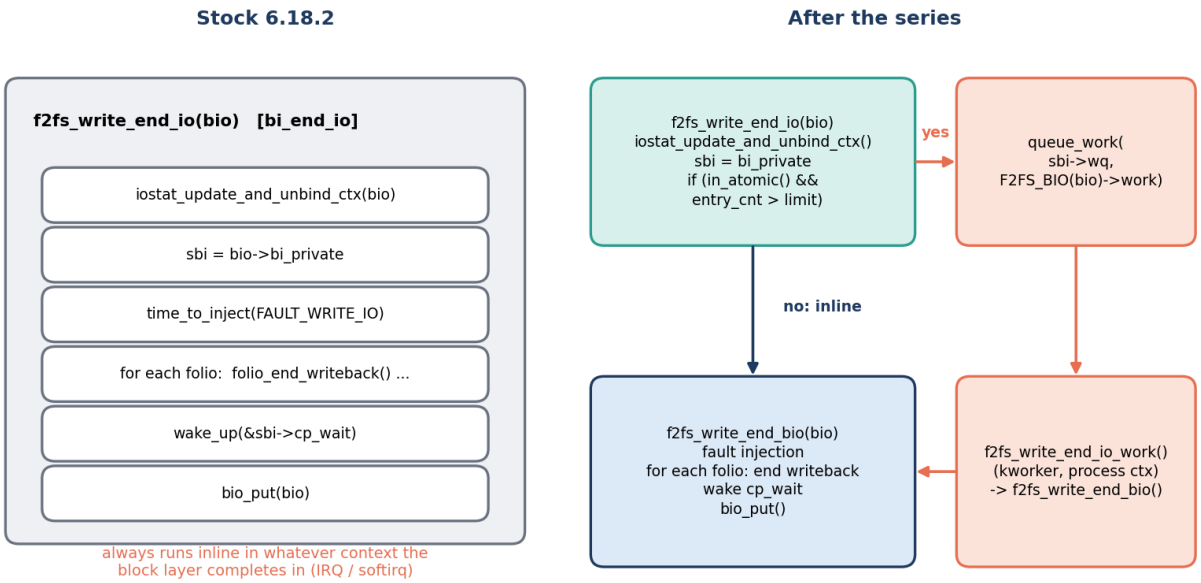


Figure 2 — Write completion before and after the series (final tree).

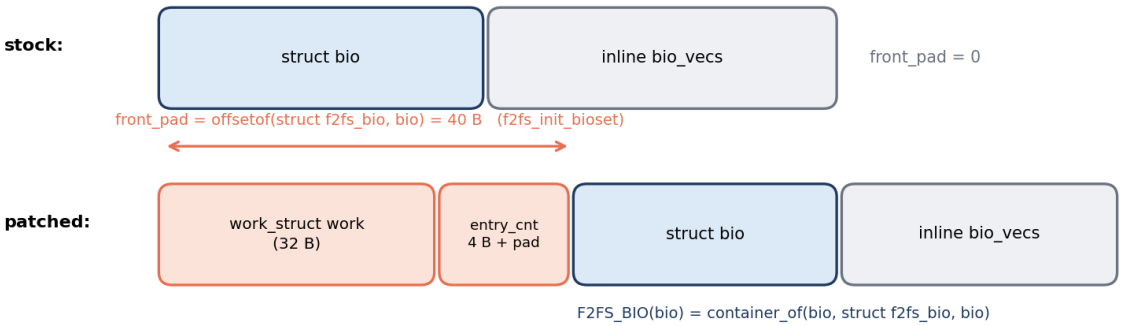


Figure 3 — Per-bio memory layout. The compiled f2fs_init_bioset() passes front_pad = 0x28 (40 bytes) on x86-64.

Net effect per file (patches 1–4 combined, against stock 6.18.2):

File	+/−	Net change
fs/f2fs/compress.c	+1 / −1	decompress free path queues to `sbi->wq`
fs/f2fs/data.c	+57 / −20	always-on `f2fs_wq` init/destroy; bioset front pad; `f2fs_write_end_bio()` / `_work()` / new `f2fs_write_end_io()`; module parameter + helper; `entry_cnt` init and 4 increments
fs/f2fs/f2fs.h	+20 / −3	`struct f2fs_bio` (work, entry_cnt, bio) + `F2FS_BIO()`; `sbi->wq`; renamed prototypes
fs/f2fs/super.c	+4 / −4	calls to `f2fs_init_wq()` / `f2fs_destroy_wq()`, new error text
sysfs.c, ABI doc	0	added in patch 2 and removed in patch 4 — no net change

5. Deviations from the upstream series

1. **No metadata-cache code**: no `max_write_bio_entry_cnt`, `f2fs_is_cache_bio()`, `f2fs_submit_cache_write()` hunk or `entry` member. `f2fs_max_write_bio_cnt()` keeps its shape so the cache series can be added later with a small diff.
2. **Patches 1–2 are reconstructions**, not Bart Van Assche's commits: names (`sbi->wq`, `f2fs_init_wq`) are inferred, the sysfs/ABI text is copied from the removal hunk of 2/2, and details that I could not see (e.g. any extra upstream changes in that series) are absent.
3. **Workqueue always allocated** (6.18.2 created it only for encrypt/verity/compression).

4 **Bioset front pad** now 40 bytes for read *and* write bios.

5 **Counting sites** use 6.18.2's page-based `bio_add_page()` context in `add_bio_entry()` / `add_ipu_page()`; there is no `bi_write_stream` line in `__bio_alloc()`.

6 **ABI entry** is appended at the end of the file in patch 2 (upstream has it mid-file) and removed in patch 4.

7 **Patch 3 subject** shortened to "…max_write_bio_folio_cnt module parameter" because only that parameter exists.

6. Verification

Check	How	Result
Series applies in order	`git am --no-3way` of the 4 patches onto the pristine 6.18.2 tree	**OK** — no fuzz, no offsets
Combined diff applies	`patch -p1 --dry-run` on pristine fs/ + Documentation/	**OK** (touches 4 files)
Result identical across re-applies	compare the tree of the compiled branch with a fresh re-apply of the final patch files	**identical** (0 diff lines)
Compiles	gcc 13.3, x86-64, W=1, `CONFIG_PREEMPT`; with F2FS compression (LZO/LZ4/ZSTD), xattr/ACL/security, IOSTAT, FAULT_INJECTION, CHECK_FS, STAT_FS, BLK_DEV_ZONED, FS_ENCRYPTION, FS_VERITY	**OK** — no warnings; objects only (no full link)
Symbols / parameter	`nm`, `strings`, `objdump` on data.o	`f2fs_write_end_io_work` present; `f2fs.max_write_bio_folio_cnt` in modinfo; `front_pad` = 0x28
All write-bio sites counted	grep audit of `bi_end_io`, `bio_alloc*`, `bio_add_*` over fs/f2fs	1 allocation site (`__bio_alloc`) and 4 add sites, all covered; other bio_alloc users are read or discard paths

Limits of the verification

Not verified: no boot, no xfstests, no fault-injection/unmount-race runs, no UFS or device-mapper testing, and no comparison with the real upstream async-completion patches. The compile used a config derived from the host's 6.18 config with a hand-generated autoconf.h (flex/bison were unavailable). A runnable test plan for a real LUN is provided separately (f2fs-bio-cnt-tests).

7. Behavioural impact, risks and follow-ups

1 **Default behaviour is unchanged**: limit = UINT_MAX, so nothing is offloaded and completion stays inline exactly as in stock 6.18.2.

2 **Fixed costs now present:** one unbound high-priority workqueue per mount; 40 B front pad per bio; one increment per folio added; one compare per completion.

3 **Offload trigger is in_atomic()**: it depends on preempt-count tracking in the kernel config and does not detect every non-sleepable context. Check with test F03 that the path really triggers on your platform.

4 **Completion order:** `iostat_update_and_unbind_ctx()` still runs in the original context; only the folio loop, `cp_wait` wake-up and `bio_put()` move to the worker. The zoned wrapper still calls `complete(&io->zone_wait)` before the dispatcher.

5 **Unmount safety:** pending completion work is drained by `destroy_workqueue()` in `f2fs_put_super()`, before the sbi is freed. 6.18.2 predates an upstream fix for a possible use-after-free of sbi at the tail of the completion path (early 2026); deferring to a worker widens that window slightly — consider backporting that fix. Test F05 targets this.

6 **ABI change:** after patch 4, scripts that write /sys/fs/f2fs/<disk>/max_atc_write_bio_size get ENOENT. That attribute never existed in stock 6.18.2, so only trees that already carry it are affected.

7 **Divergence from upstream:** patches 1–2 will conflict with the real async-completion series; resolve by taking upstream and re-applying patches 3–4.

8. Deliverables

File	Content
`0001-f2fs-rename-f2fs_post_read_wq-to-f2fs_wq-and-always-.patch`	patch 1 (reconstructed)
`0002-f2fs-Run-f2fs_write_end_io-asynchronously.patch`	patch 2 (reconstructed)
`0003-f2fs-introduce-max_write_bio_folio_cnt-module-parame.patch`	patch 3 (adapted 1/2)
`0004-f2fs-remove-max_atc_write_bio_size.patch`	patch 4 (adapted 2/2)
`combined-f2fs-6.18.2-backport.diff`	all four as one diff (net change)
`README.md`	short notes (same content as §5–§6)

```
cd linux-6.18.2
git am 000*.patch                # or: patch -p1 < combined-f2fs-6.18.2-backport.diff
# enable with:  f2fs.max_write_bio_folio_cnt=N   (kernel cmdline or modprobe option; default UINT_MAX = off)
```